



Digital System Design

DSD Final Project

Hardware Implementation of Pipelined RISC-V CPU

Speaker: Pony

Mentor: Ben

Instructor: 吳安宇教授

Date: 2026/05/07



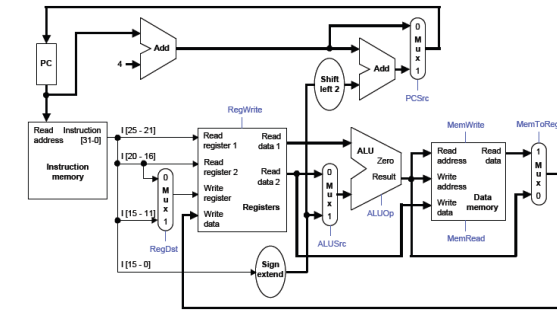
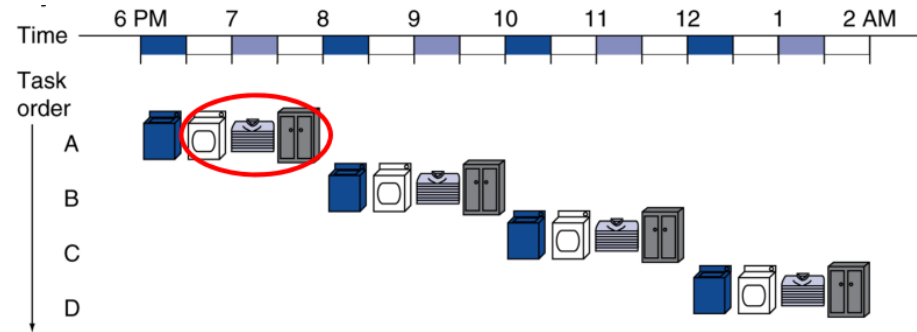
RISC-V Processors

❖ Single-Cycle CPU

❖ HW2

❖ Low throughput

❖ Simple design



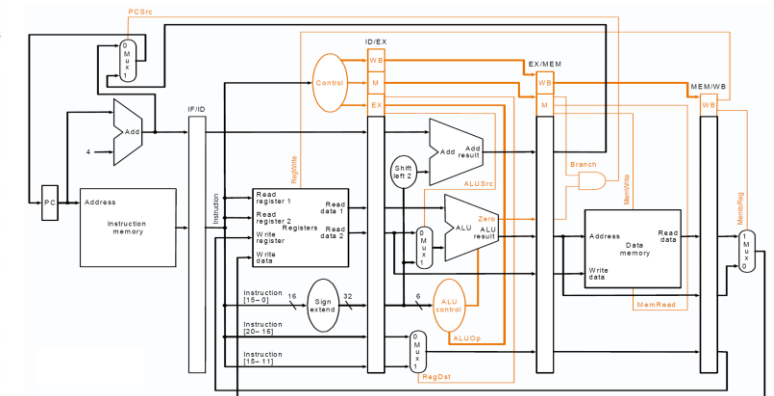
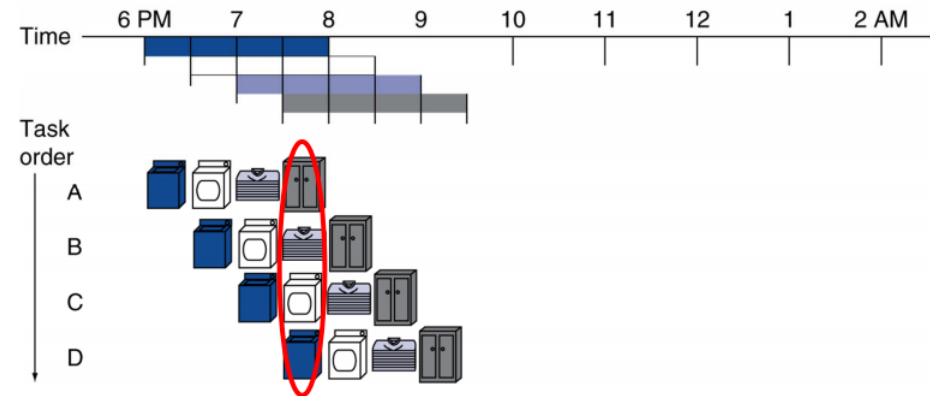
❖ Pipelined CPU

❖ Final Project

❖ Higher throughput

❖ Complex design

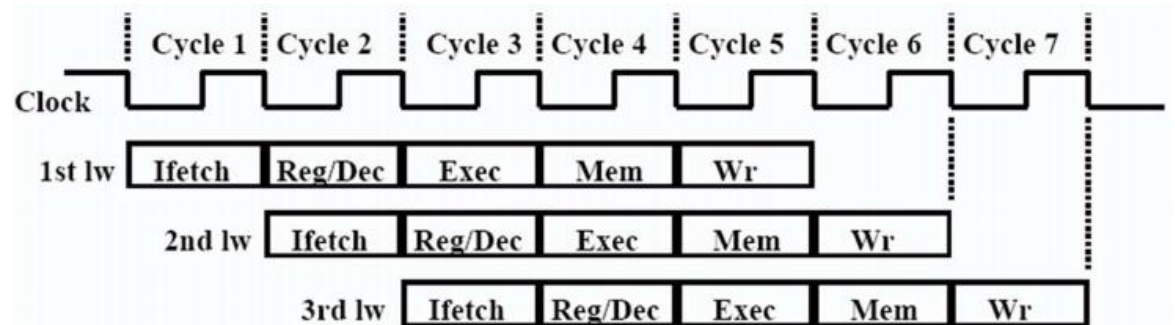
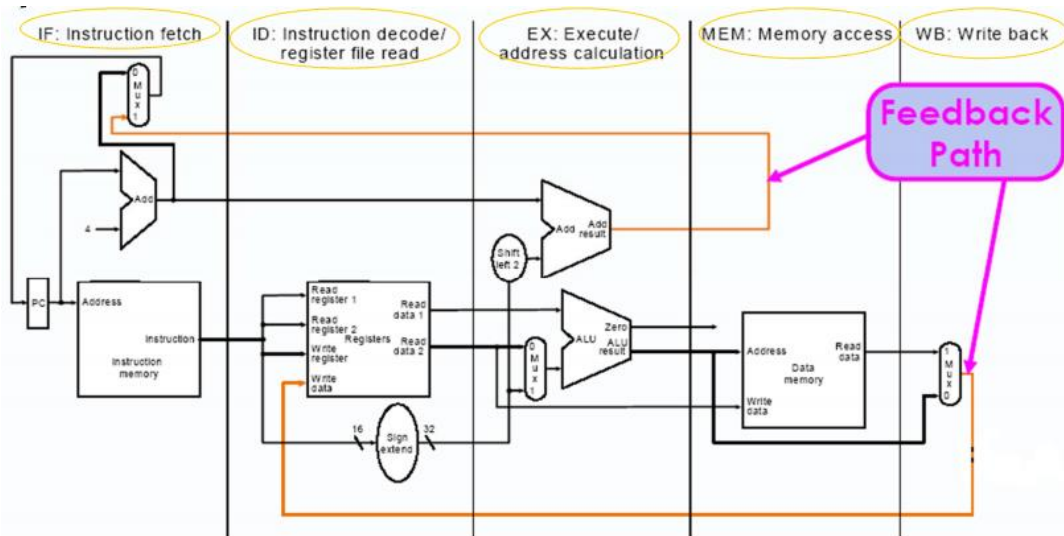
➤ For handle hazard





Final Project: Pipelined RISC-V Processor

- ❖ In this project, you should implement a hardware design of pipelined RISC-V CPU with **more than 5 pipeline stages** while the **clock period should not exceed 3.5ns**
- ❖ Splits into several functional unit and perform instruction in parallel way



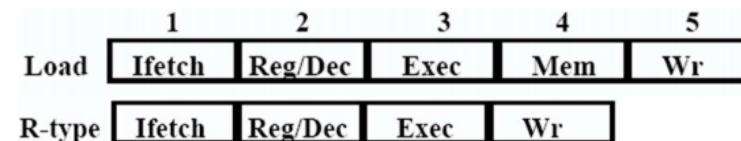


Hazards

❖ In order to implement pipelined RISC-V CPU, you need to resolve potential hazards in your hardware design

❖ Structural Hazard

➤ The hardware can't support the combination of instructions



❖ Data Hazard

➤ Data that is needed to execute the instruction is not yet available (dynamic).

❖ Load-use Data Hazard

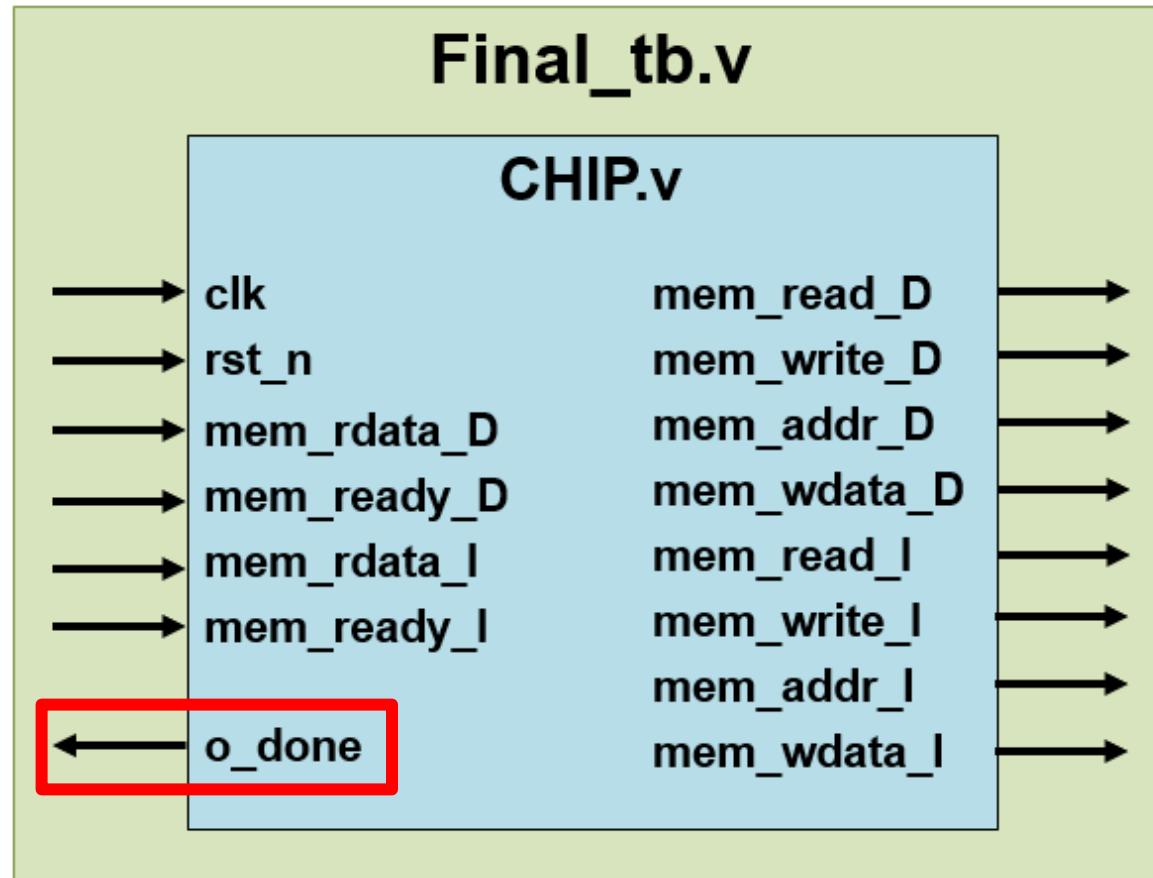
➤ Load instruction (lw) that reads data from main memory has not yet become available when it is required

❖ Control Hazard (Branch Hazard)

➤ Instruction that was fetched is **NOT** the one that is needed



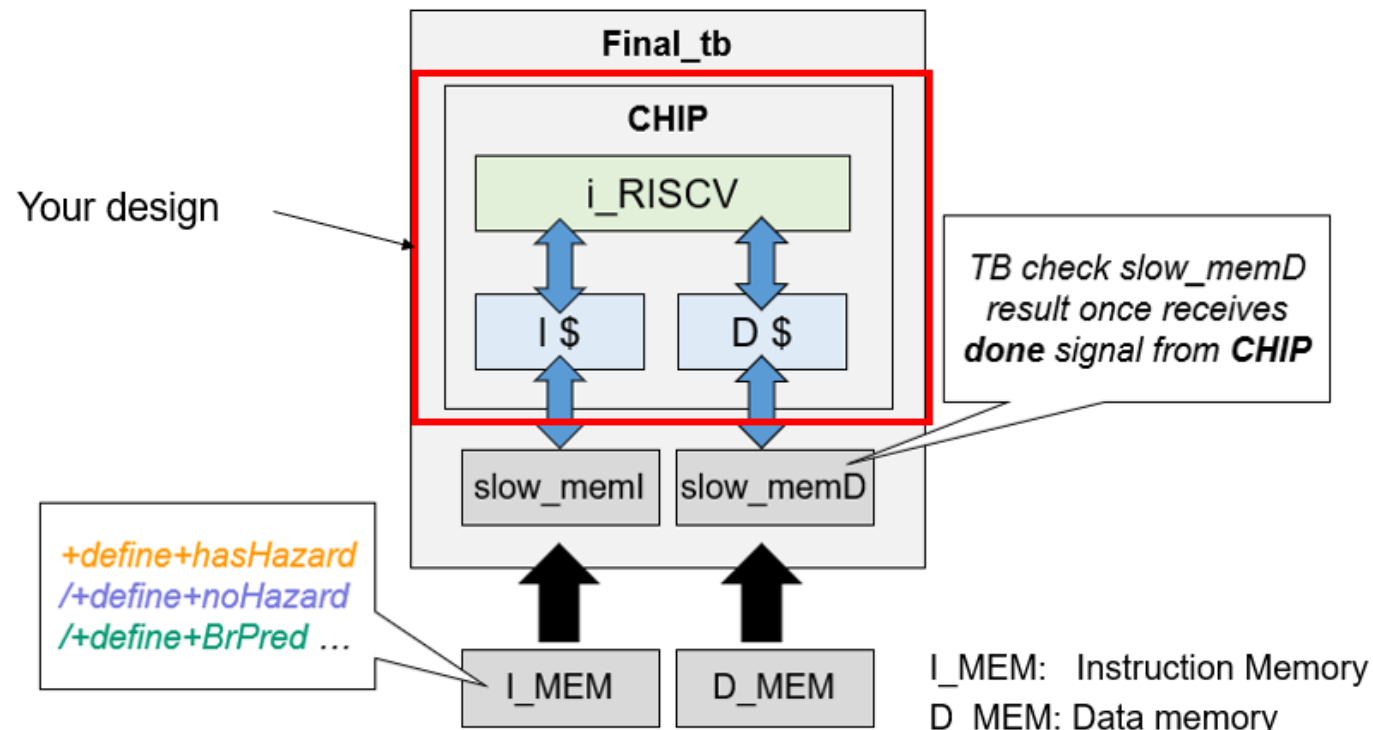
Block Diagram





How does the test work?

- ❖ The cache is optional, but is highly recommended to improve the performance
- ❖ The testbench will compare the data in **slow_memD** with the golden answer once it receives **o_done** signal from CHIP





Memories

- ❖ Your design should pass with all 3 types of memories

Memory type	Behavior
fast_memory.v	No additional stall
slow_memory.v	Fixed latency stall
slow_memory_random_latency.v	Random latency stall

- ❖ The final performance will be tested with **slow_memory_random_latency.v**
 - ❖ The seed will be set to **32'h1142D5D** for fair comparison
 - ❖ The random latency will be set to **[40, 50] ns**
- ❖ To change the type of memories used in simulation, modify the definition in the file **00_TESTBED/testbench/tb_define.v**



Required Instruction Set

- ❖ You need to modify several parts to fit our specifications.
- ❖ For example, you need to add the path for **J-type instructions**
- ❖ Note that an extra instruction **flush** is required if there's cache in your design

5.2. cbo.flush

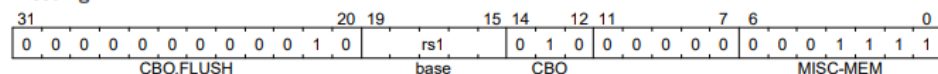
Synopsis

Perform a flush operation on a cache block

Mnemonic

cbo.flush offset(*base*)

Encoding



Description

A **cbo.flush** instruction performs a flush operation on the cache block whose effective address is the base address specified in *rs1*. The offset operand may be omitted; otherwise, any expression that computes the offset shall evaluate to zero. The instruction operates on the set of coherent caches accessed by the agent executing the instruction.

Table 1. Required Instruction Set

Name	Description
ADD	Addition, overflow detection for signed operand is not required*
ADDI	Addition immediate with sign-extension, without overflow detection*
SUB	Subtract, overflow detection for signed operand is not required*
AND	Boolean logic operation
ANDI	Boolean logic operation with 12bit of immediate
OR	Boolean logic operation
ORI	Boolean logic operation with 12bit of immediate
XOR	Boolean logic operation
XORI	Boolean logic operation with 12bit of immediate
SLLI	Shift left logical (zero padding)
SRAI	Shift right arithmetic (sign-digit padding)
SRLI	Shift right logical (zero padding)
SLT	Set less than, comparison instruction
SLTI	Set less than variable, comparison instruction
BEQ	Branch on equal, conditional branch instruction
BNE	Branch on not equal, conditional branch instruction
JAL	Unconditionally jump and link (Save next PC in \$rd)
JALR	Jump and link register(Save next PC in \$rd)
LW	Load word from data memory (assign word-aligned)
SW	Store word to data memory (assign word-aligned)
NOP	No operation(addi \$r0 \$r0 0)

* Different from definition in [1], the exception handler for arithmetic overflow is not required.



Instruction to Flush Cache Data

- ❖ For simplicity, we set the block index to 0 but synchronize all cache data to the memory if the instruction is received.

```
`define INST_FLUSH 32'h07_20_20_00
```

- ❖ Once the operation is done, the **o_done** signal should be pulled to high. The testbench will then check the memory result.
- ❖ If you don't have caches in your design, the flush instruction works as **NOP**
- ❖ The flush instruction is guaranteed to be the last instruction in each pattern. (There will be only NOP instructions after the flush instruction)



EXTENSIONS



Extensions

❖ Branch Prediction

- ❖ Always not branch
- ❖ Branch / Not branch / Branch
- ❖ Always branch

❖ Supporting compressed instructions

- ❖ Extract C-instructions

❖ Supporting multiplication instructions

- ❖ Implement multiplication unit



Extension: Branch Prediction

❖ For loop is commonly see in programming

```
for (i = 0 ; i < k ; i ++ ) {
    // Loop function
}
```

They do **BEQ** in every loop

❖ To increase overall efficiency of system, branch prediction is always used

ADD \$t0, \$0, \$0	#set \$t0 to 0	for(int i=0; i != 100; i++)	{...}
ADDI \$t1, \$0, 100	#set \$t1 to 100	for(int i=0; i != 100; i++)	{...}
SLL...	#loop body	for(int i=0; i != 100; i++)	{...}
ADD...	#assume this loop contains 3 instructions		
SUB...			
ADDI \$t0, \$t0, 1	#add 1 to \$t0	for(int i=0; i != 100; i++)	{...}
BNE \$t0, \$t1, -5	#branch if \$t0 != \$t1	for(int i=0; i != 100; i++)	{...}

Waste only 3 cycles with branch prediction

Waste 99 cycles if processing normally



Compressed Instruction

- ❖ Implement the following 16 compressed instructions as the extension to your base RISC-V core

C.ADD	C.ANDI	C.LW	C.J
C.MV	C.SLLI	C.SW	C.JAL
C.ADDI	C.SRLI	C.BEQZ	C.JR
C.NOP	C.SRAI	C.BNEZ	C.JALR

- ❖ Extract information encoded in C-instructions
- ❖ Some adjustment must be applied
 - ❖ PC increment
 - ❖ Address alignment especially for the design with cache



Multiplication Instruction

- ❖ Implement the following **Multiplication** instruction to your base RISC-V core

RV32M Standard Extension					
0000001	rs2	rs1	000	rd	0110011
MUL					

- ❖ Design multiplication unit
- ❖ Timing optimization to pass the clock period requirement
 - ❖ Pipelined multiplication
 - ❖ Parallelization
- ❖ Hazard issues may need to be solved



Simulation with Final_tb (1/2)

- ❖ Final_tb.v in 00_TESTBED/testbench/
- ❖ You can find more details in 00_TESTBED/testbench/pat_define.v

```
// =====  
// Baseline Patterns  
// =====  
`ifdef noHazard  
  `define N_MEM_CHECK      256  
  `define IMEM_INIT        "../00_TESTBED/pattern/Hazard_None/I_mem_noHazard"  
  `define DMEM_INIT        "../00_TESTBED/pattern/Hazard_None/D_mem"  
  `define GOLDEN            "../00_TESTBED/pattern/Hazard_None/D_gold"  
`endif  
  
`ifdef hasHazard  
  `define N_MEM_CHECK      256  
  `define IMEM_INIT        "../00_TESTBED/pattern/Hazard/I_mem_hasHazard"  
  `define DMEM_INIT        "../00_TESTBED/pattern/Hazard/D_mem"  
  `define GOLDEN            "../00_TESTBED/pattern/Hazard/D_gold"  
`endif
```

[License]

```
source /usr/cad/synopsys/CIC/vcs.cshrc  
source /usr/spring_soft/CIC/verdi.cshrc
```

[Simulation]

// RTL (simulate under 01_RTL)

```
vcs ../00_TESTBED/testbench/Final_tb.v [rtl_files.v]\  
../00_TESTBED/testbench/flush_inst_define.v\  
../00_MEMORY/slow_memory_random_latency.v -full64 -R -debug_access+all +v2k \  
+define+noHazard
```

// Gate level (simulate under 03_GATE)

```
vcs ../00_TESTBED/testbench/Final_tb.v ../02_SYN/Netlist/CHIP_syn.v\  
../00_TESTBED/testbench/flush_inst_define.v\  
../00_MEMORY/slow_memory_random_latency.v -v tsmc13.v -full64 -R \  
-debug_access+all +v2k +define+noHazard +define+SDF
```



Simulation with Final_tb (2/2)

- ❖ It's recommended to generate a **Makefile** for simplicity
- ❖ You can run the simulation with the provided Makefile by using **make run**
- ❖ To change the pattern, you can modify the pattern flag defined in 01_rtl.f and 03_gate.f
- ❖ Remember to check the contents in 01_rtl.f and 03_gate.f before you run the simulation

```
#####  
# DUT #  
#####  
CHIP.v
```

```
#####  
# Pattern Definition #  
#####  
+define+noHazard  
// +define+hasHazard  
  
// +define+BrPred  
// +define+Scaling  
// +define+compression  
// +define+compression_uncompressed  
  
// +define+QSort_uncompressed  
// +define+QSort  
// +define+Conv  
// +define+Conv_uncompressed  
// +define+Mul  
  
// +define+LFSR_HIST  
// +define+LFSR_HIST_short
```



Synthesis

- ❖ You should identify suitable input / output delay based on the testbench.

```
#####  
set t_in 0.0  
set t_out 0.0  
set_input_delay $t_in -clock CLK [remove_from_collection [all_inputs] [get_ports clk]]  
set_output_delay $t_out -clock CLK [all_outputs]  
#####
```

- ❖ Latches are not allowed in gate level code after synthesis, use Flip-flop instead.
- ❖ Cycle time in Gate-level simulation cannot be lower than the cycle time in SDC file even if no violations occur.



Grading Policy

❖ The score of this project consists of two aspects

❖ 1) Baseline checkpoint (40%)

- Checkpoint presentation (5%)
- Gate level simulation pattern **noHazard** (15%)
- Gate level simulation pattern **hasHazard** (20%)

❖ 2) Final presentation and submission (60%)

- Final Presentation (8%)
- Branch Prediction Gate Level (9%)
- Compressed Instr. Gate Level (9%)
- Multiplication Instr. Gate Level (9%)
- Q_sort & Conv & LFSR_HIST Performance Ranking (15%)
- Report (10%)



Baseline Checkpoint (40%)

❖ Checkpoint

- ❖ Deliver presentation with 4-6 pages slides (about 5 minutes)
- ❖ Confirm your current results and future plan

❖ Pass test programs (pattern **noHazard** and **hasHazard**)

- ❖ Supporting all instructions mentioned before
- ❖ Complete the circuit synthesis with tsmc13 process
- ❖ The slack cannot be negative
- ❖ The reported period should not be less than the period specified in synthesis stage
- ❖ **Your design must have more than 5 pipeline stages and the period must below 3.5ns**



Final Presentation and Submission (60%)

- ❖ Each team should prepare for a full presentation for your fantastic work on extension! (**about 10~15 slides within 10 minutes**)
- ❖ Implement as much and deep as you can of the topics of extensions
 - ❖ Deeper exploration or more ablation study lead to higher score
 - ❖ The content also affect quality of presentation and report
- ❖ The performance is evaluated by pattern Q_sort, Conv and LFSR_HIST

$$A \times T_{Q_sort} \times T_{Conv} \times T_{LFSR_HIST}$$

A: Area (um²)

T_{Q_sort} : Simulation time of Q_sort (ns)

T_{Conv} : Simulation time of Conv (ns)

T_{LFSR_HIST} : Simulation time of LFSR_HIST (ns)



※ Notice

- ❖ Latches are not allowed in gate level code after synthesis, use Flip-flop instead.
- ❖ Negative Slack and Timing Violations are not allowed after synthesis.
- ❖ There will be **hidden cases** for all test patterns.
- ❖ **Cycle time in Gate-level simulation cannot be lower than the cycle time in SDC file.**
- ❖ **You should not assume memory size is 1024 x 128 bit.**
- ❖ PC address should be reset to 0.
- ❖ You should identify suitable input / output delay based on the testbench.
- ❖ The behavior of memory is different from the memory in HW3.



Schedule

Date	Submission/Event
5/07	Final project announcement
5/28	A. Baseline checkpoint. Each team should prepare a presentation (<i>4-6 pages, about 5 minutes</i>) to confirm your current results and future plan. B. You should <i>upload the source codes and slide to COOL before 5/27 23:59</i>. C. Briefly show your design for passing Baseline testbenches (noHazard, hasHazard). D. Extension topics plan should be included in the presentation E. Please attach the work assignment chart on last page
6/11	A. Final presentation. Each team should prepare a full talk (about 10-15 slides <i>within 10 minutes</i>) to demonstrate your fantastic work! Detail presentation plan will be announced on COOL. B. You should upload the source codes (including all the RTL code and synthesis related files: *.v, *.sdf, *.ddc and your performance) to COOL before <i>6/10 23:59</i>.
6/14	Report submission. You should upload the report (including a detailed report (<i>8-16 pages</i>) and the presentation slides). You should upload them to COOL before <i>6/14 23:59</i> .



THANKS FOR LISTENING